

2. Offline Text-to-Speech (TTS)

2. Offline Text-to-Speech (TTS)

1. Concept Introduction

1.1 What is "TTS"?

1.2 Brief Description of the Principle

1. Text Analysis

2. Language Processing

3. Speech Synthesis

4. Sound Waveform Generation

2. Code Analysis

Key Code

1. TTS Initialization and Calling (`large_model/large_model/model_service.py`)

2. TTS Backend Implementation (`large_model/utils/large_model_interface.py`)

Code Analysis

3. Practical Operation

3.1 Configure Offline TTS Function

3.2 Start and Test the Function

4. Common Problems and Solutions

4.1 Playback Issues

Problem 1: The program runs normally without errors, but no sound is heard.

1. Concept Introduction

1.1 What is "TTS"?

TTS technology is a technology that converts written text into human-audible speech output. It enables computers to "read" text content and is widely used in accessible reading, smart assistants, navigation systems, educational software, and many other fields. Through TTS, users can hear naturally fluent human voices generated by machines, greatly enhancing the convenience and flexibility of information acquisition.

1.2 Brief Description of the Principle

The implementation of a TTS system mainly includes the following key steps and technologies:

1. Text Analysis

- In this stage, the input text is first preprocessed, including but not limited to removing irrelevant characters, normalizing punctuation and capitalization, word segmentation, and identifying special symbols such as numbers and converting them into corresponding word forms.
- At the same time, linguistic analysis of the text is also required, such as determining the pronunciation of each word (which usually requires the use of a pronunciation lexicon), stress position, intonation pattern, and sentence structure.

2. Language Processing

- This step mainly focuses on how to pronounce and adjust the intonation correctly according to the context. For example, the word "read" has different pronunciations in different tenses (the past tense/past participle is pronounced /red/, while in other cases it is pronounced /ri:d/). Therefore, a powerful language model is needed to understand these nuances.
- This also involves prosody modeling, which is to decide which parts should be emphasized, whether the speech rate should be fast or slow, and the emotional tone of the entire sentence.

3. Speech Synthesis

- The text information processed in the previous two stages is sent to a speech synthesis engine, which is responsible for generating the actual sound waveform.
- Traditional TTS systems use concatenative synthesis, which is to select appropriate units from a pre-recorded speech segment database and concatenate them to form a complete sentence. Although this method can produce high-quality speech, it is limited by the samples in the database.
- Modern TTS systems rely more on parametric synthesis or neural network synthesis (such as WaveNet, Tacotron, etc.). These methods can directly predict speech features from text and generate continuous sound signals. In particular, deep learning-based methods can better capture the subtle changes in speech, thereby producing more natural and fluent sounds.

4. Sound Waveform Generation

- Finally, the generated sound waveform is further processed to ensure that its quality meets the expected standards, such as adjusting the volume, equalizing the frequency response, etc.
- After that, this audio data can be played out through speakers or other audio playback devices for people to listen to.

With the development of artificial intelligence and machine learning technologies, especially the application of deep learning, TTS systems have not only significantly improved in accuracy, but also made great progress in naturalness and emotional expression, making machine-generated speech closer and closer to real human voices.

2. Code Analysis

Key Code

1. TTS Initialization and Calling

(`largemodel/largemodel/model_service.py`)

```
# From largemodel/largemodel/model_service.py
class LargeModelService(Node):
    def __init__(self):
        # ...
        self.system_sound_init()
        # ...

    def init_param_config(self):
        # ...
```

```

        self.declare_parameter('useolinetts', False)
        self.useolinetts =
self.get_parameter('useolinetts').get_parameter_value().bool_value
        if self.useolinetts:
            self.tts_out_path = os.path.join(self.pkg_path, "resources_file",
"tts_output.mp3")
        else:
            self.tts_out_path = os.path.join(self.pkg_path, "resources_file",
"tts_output.wav")

def system_sound_init(self):
    """Initialize TTS system"""
    model_type = "oline" if self.useolinetts else "local"
    self.model_client.tts_model_init(model_type, self.language)
    self.get_logger().info(f'TTS initialized with {model_type} model')

def _safe_play_audio(self, text_to_speak: str):
    """
    Synthesizes and plays all non-empty messages only in non-text chat mode.
    """
    if not self.text_chat_mode and text_to_speak:
        try:
            self.model_client.voice_synthesis(text_to_speak,
self.tts_out_path)
            self.play_audio_async(self.tts_out_path)
        except Exception as e:
            self.get_logger().error(f"Safe audio playback failed: {e}")

```

2. TTS Backend Implementation

(`largemodel/utis/large_model_interface.py`)

```

# From largemodel/utis/large_model_interface.py
class model_interface:
    # ...
    def tts_model_init(self, model_type='oline', language='zh'):
        if model_type=='oline':
            if self.tts_supplier=='baidu':
                self.token=self.fetch_token()

            self.model_type='oline'
        elif model_type=='local':
            self.model_type='local'
            if language=='zh':
                tts_model=self.zh_tts_model
                tts_json=self.zh_tts_json
            elif language=='en':
                tts_model=self.en_tts_model
                tts_json=self.en_tts_json
            self.synthesizer = piper.PiperVoice.load(tts_model,
config_path=tts_json, use_cuda=False)

    def voice_synthesis(self, text, path):
        if self.model_type=='oline':
            if self.tts_supplier=='baidu':
                # ... (Baidu TTS implementation)

```

```

        pass
    elif self.tts_supplier=='aliyun':
        # ... (Aliyun TTS implementation)
        pass
    elif self.model_type=='local':
        with wave.open(path, 'wb') as wav_file:
            wav_file.setnchannels(1)
            wav_file.setsampwidth(2)
            wav_file.setframerate(self.synthesizer.config.sample_rate)
            self.synthesizer.synthesize(text, wav_file)

```

Code Analysis

The Text-to-Speech (TTS) function is initiated by the `LargeModelService` node and implemented by the `model_interface` class. Its design allows switching between different backend services through parameter configuration.

1. Initialization Process (`model_service.py`):

- During the initialization of `LargeModelService`, the `init_param_config` function reads the boolean value `use_online_tts` from the ROS parameter server.
- The `system_sound_init` function passes either a `'local'` or `'online'` string to the `self.model_client.tts_model_init` method based on the value of `use_online_tts`.
- In `large_model_interface.py`, the `tts_model_init` method executes the corresponding initialization logic based on the received string parameter. If it is `'local'`, it loads the local model file using `piper.PiperVoice.load`.

2. Synthesis and Playback Process (`model_service.py`):

- When voice playback is needed, the `_safe_play_audio` function is called.
- This function first calls the `self.model_client.voice_synthesis` method, passing in the text to be converted and the target audio path `self.tts_out_path`.
- After the `voice_synthesis` method finishes executing and generates the audio file, `_safe_play_audio` then calls `self.play_audio_async` to play the file asynchronously.

3. Backend Implementation Selection (`large_model_interface.py`):

- The `voice_synthesis` method is the backend distribution center for the TTS function. It internally selects the execution path by checking the value of the `self.model_type` attribute set during initialization.
- If `self.model_type` is `'local'`, the code block opens a WAV file using Python's `wave` library, sets its header parameters (channels, sample width, frame rate), and then calls the `self.synthesizer.synthesize` method to write the synthesized audio stream directly to the file.
- If `self.model_type` is `'online'`, it will enter the logic branches for different cloud service providers (such as Baidu, Aliyun).
- This structure separates the upper-level node's call ("speak this sentence") from the lower-level specific synthesis technology (which library to use, which API to call).

3. Practical Operation

3.1 Configure Offline TTS Function

To enable offline TTS, you need to correctly configure `yahboom.yaml` and `large_model_interface.yaml`, and ensure that the local model is placed correctly.

1. Open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify/Confirm the following key configurations:

```
model_service:                                # Model server node parameters
  ros__parameters:
    language: 'en'                             # Large model interface language
    useonetts: False                           # whether to use online speech
    synthesis (True for online, False for offline)
    regional_setting : "China"
```

`useonetts` must be `False` to use the local model.

The language selection `zh` is for Chinese and `en` is for English.

3. Open the model interface configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

4. Confirm the offline model path:

```
# large_model_interface.yaml
## offline TTS
# Chinese TTS model
zh_tts_model: "/root/yahboom_ws/src/largemodel/MODELS/tts/zh/zh_CN-huayan-
medium.onnx"
zh_tts_json: "/root/yahboom_ws/src/largemodel/MODELS/tts/zh/zh_CN-huayan-
medium.onnx.json"

# English TTS model
en_tts_model: "/root/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-libritts-
high.onnx"
en_tts_json: "/root/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-libritts-
high.onnx.json"
```

3.2 Start and Test the Function

1. Start the TTS node:

Run the following command:

```
ros2 launch largemodel tts_only.launch.py
```

```
~/yahboom_ws$ ros2 launch largemodel tts_only.launch.py
[INFO] [launch]: All log files can be found below /home/sunrise/.ros/log/2025-08-07-16-44-02-726089-ubuntu-33939
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [tts_only-1]: process started with pid [33940]
[tts_only-1] [INFO] [1754556253.838432585] [tts_only_node]: TTS Only Node is starting...
[tts_only-1] [INFO] [1754556253.840804978] [tts_only_node]: Language set to: zh
[tts_only-1] [INFO] [1754556253.841469608] [tts_only_node]: Using online TTS: False
[tts_only-1] [INFO] [1754556253.842111438] [tts_only_node]: TTS output path: /home/sunrise/yahboom_ws/install/largemodel/share/largemodel/resources_file/tts_output.wav
[tts_only-1] [INFO] [1754556255.723894754] [tts_only_node]: TTS initialized with local model
```

2. Send the text to be synthesized:

Open a new terminal and run the following command to publish a voice broadcast message:

```
ros2 topic pub --once /tts_text_input std_msgs/msg/String '{data: "speech synthesis test successful"}'
```

3. Test:

If everything is normal, you should hear the robot say "Speech synthesis test successful" from your speaker in a synthesized voice.

4. Common Problems and Solutions

4.1 Playback Issues

Problem 1: The program runs normally without errors, but no sound is heard.

Solution:

1. **Check the audio output:** Make sure your system's audio output device is selected correctly and the volume is not muted. You can try playing a normal music file to test the hardware.